

VISIÓN 3D (14536)

RÚBRICAS DE EVALUACIÓN

Rúbricas de Evaluación para la Asignatura de Visión 3D

A. Observación (20%) -> $A = a+b$

- a) Participación en clase, solución de problemas de aula, prácticas (10%)
- b) Presentación oral del proyecto (10%)

B. Proyecto 1 (40%) -> $B = (c+d+e)*f$

- c) Organización (10%)
- d) Implementación Técnica (20%)
- e) Resultados y Análisis (10%)
- f) Coevaluación (0-1)

C. Proyecto 2 (40%) -> $B = (g+h+i)*j$

- g) Organización (10%)
- h) Implementación Técnica (20%)
- i) Resultados y Análisis (10%)
- j) Coevaluación (0-1)

Rúbricas de Evaluación para la Organización (10%)

- Organización del trabajo en equipo (5%)
- Documentación (5%)
 - o **Excelente (5):** La documentación es completa, clara y bien organizada, facilitando la comprensión del proyecto.
 - o **Bueno (4):** La documentación es adecuada y bien organizada, con información suficiente.
 - o **Satisfactorio (3):** La documentación es básica, con información general y organización adecuada.
 - o **Insuficiente (0-2):** La documentación es incompleta, confusa o desorganizada.

Rúbricas de Evaluación para la Implementación Técnica (20%)

- Calidad del Código (5%)
 - o **Excelente (5):** El código es limpio, bien comentado y sigue las mejores prácticas de programación.
 - o **Bueno (4):** El código es adecuado, con comentarios suficientes y buenas prácticas de programación.
 - o **Satisfactorio (3):** El código es funcional, pero con comentarios limitados y algunas prácticas de programación mejorables.

- o **Insuficiente (0-2):** El código es confuso, mal comentado y no sigue buenas prácticas de programación.
- **Uso de Herramientas y Técnicas (15%)**
 - o **Excelente (12-15):** Utiliza herramientas y técnicas avanzadas de visión 3D obteniendo la mejor precisión de manera y efectiva.
 - o **Bueno (9-12):** Utiliza herramientas y técnicas adecuadas de visión 3D.
 - o **Satisfactorio (7-9):** Utiliza herramientas y técnicas básicas de visión 3D.
 - o **Insuficiente (0-7):** Utiliza herramientas y técnicas de manera inadecuada o limitada.

Rúbricas de Evaluación para los Resultados y el Análisis (10%)

- **Presentación de Resultados (5%)**
 - o **Excelente (5):** Los resultados son claros, bien presentados y visualmente atractivos.
 - o **Bueno (4):** Los resultados son claros y bien presentados.
 - o **Satisfactorio (3):** Los resultados son básicos y adecuadamente presentados.
 - o **Insuficiente (0-2):** Los resultados son confusos o mal presentados.
- **Análisis y Conclusiones (5%)**
 - o **Excelente (5):** El análisis es profundo y las conclusiones están bien fundamentadas.
 - o **Bueno (4):** El análisis es adecuado y las conclusiones son razonables.
 - o **Satisfactorio (3):** El análisis es básico y las conclusiones son generales.
 - o **Insuficiente (0-2):** El análisis es superficial y las conclusiones son débiles.

Los equipos de trabajo deberéis de plantearos las siguientes preguntas:

1. ¿Cómo se puede organizar el trabajo en equipo? En la asignatura se propone seguir la metodología SCRUM.
2. ¿Cómo se debe documentar un proyecto? En la asignatura se propone presentar un portfolio explicando quién ha hecho qué y las transparencias de la presentación. Es muy importante documentar el proceso de guiado de la IA, indicando las secuencias de prompts utilizadas y las correcciones de las alucinaciones.
3. ¿Cómo se debe diseñar el software? ¿Qué se considera buenas prácticas de programación? ¿Cómo se debe testear el software? En la asignatura se proponen las prácticas del siguiente apartado.
4. ¿Cómo se debe hacer una presentación? En esta asignatura se propone distribuir el Pecha Kucha en los siguientes apartados: definición del problema, antecedentes, metodologías utilizadas, soluciones propuestas, experimentos realizados, resultados y conclusión.

Buenas prácticas de diseño, programación y test

En este apartado se presentan una serie de consejos y técnicas de programación que pueden potenciar la legibilidad y la eficacia de un diseño. Al aplicar estas estrategias de manera efectiva, los desarrolladores pueden crear software más robusto, mantenible y escalable. Concretamente se presentan algunos consejos para hacer que el diseño y el código sean más comprensibles tanto para su creador como para otros programadores, tales como la inclusión de comentarios, las convenciones de nomenclatura, así como las convenciones de modelado.

Comentarios

Se sugiere que los comentarios sean breves y se enfoquen a describir las ideas principales y generales antes del código, en vez de explicar cada línea individualmente. Cada programador puede desarrollar su propio estilo para escribir sus comentarios.

Pautas de nomenclatura

Existen una serie de pautas que establecen cómo nombrar diferentes elementos del código, como variables, funciones y clases. Aunque es posible desarrollar convenciones de nomenclatura personalizadas, es recomendable adoptar las que ya son ampliamente utilizadas por la comunidad de desarrolladores.

Un consejo valioso a la hora de crear nombres para identificadores (variables, constantes, funciones, clases, etc.) es optar por un nombre que sea a la vez conciso y descriptivo. Esto se puede lograr combinando varias palabras separadas por guiones bajos o utilizando la convención de camel case, donde cada palabra comienza con una letra mayúscula. Por ejemplo, si se está implementando un contador para filas en una imagen, es mejor utilizar un nombre como "`contadorFilasImagen`" o "`fila_contador`" en lugar de algo breve pero poco descriptivo como "`tmpI`".

Lenguaje Unificado de Modelado (UML)

Lenguaje Unificado de Modelado (UML) es un conjunto de reglas para visualizar diferentes aspectos del diseño de ingeniería. Estas reglas se describen, por ejemplo, en el libro de Booch [1], que consta de aproximadamente 500 páginas. En este apartado se presenta brevemente los conceptos básicos de UML, especialmente en el contexto del diseño de sistemas de visión.

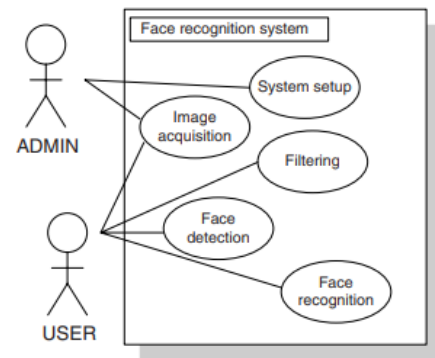
El concepto fundamental de UML es el diagrama. A continuación, se presenta una lista con los diagramas principales, junto con una breve explicación. Para obtener más información sobre el tema, se puede consultar el libro de Booch [1].

1. **Diagrama de casos de uso** - muestra las relaciones entre los llamados actores y casos de uso en un sistema. Se utilizan para modelar el comportamiento de un sistema, subsistema o clase. Cada uno presenta casos de uso con actores, es decir,

entidades participantes, así como las relaciones entre ellos. Con frecuencia se utilizan para presentar:

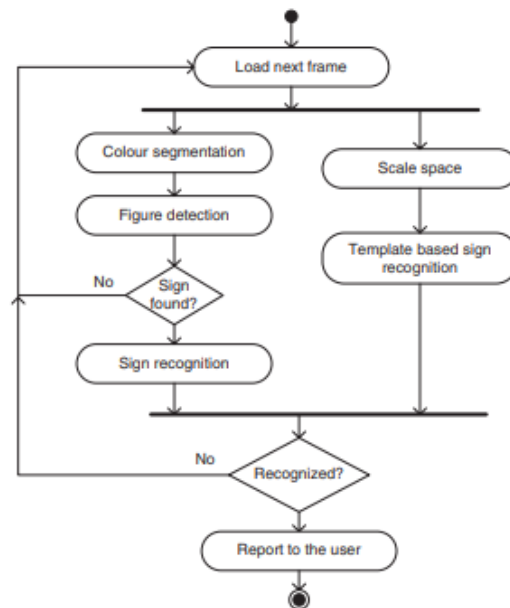
- El entorno del sistema;
- Los requisitos del sistema.

En este ejemplo, se muestra un diagrama de caso de uso de un sistema de visión para reconocimiento facial. Este modelo presenta los requisitos de ese sistema. Hay dos actores: administrador (ADMIN) y usuario (USER). Los casos de uso están colocados en el rectángulo adyacente. Estos son: configuración del sistema, adquisición de imágenes, filtrado, detección de rostros y reconocimiento facial.



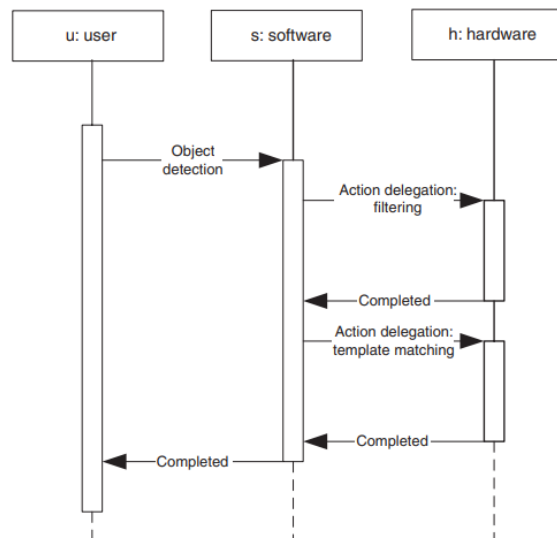
2. **Diagrama de actividades** - modela un flujo procedural del comportamiento en un sistema. Los diagramas de actividades se utilizan para modelar la dinámica de un sistema. Por lo general, es un diagrama de flujo de pasos computacionales secuenciales, pero a veces también paralelos, necesarios para cumplir una tarea determinada.

En este ejemplo, se muestra un diagrama de actividades de un sistema para reconocimiento de señales de tráfico. Las cajas ovaladas denotan pasos de acción individuales. La sincronización o división de acciones se representa con barras gruesas. Hay dos etapas de reconocimiento paralelas. La rama izquierda realiza la detección de figuras seguida del reconocimiento de señales. La rama derecha realiza un reconocimiento de patrones en el espacio log-polar. Por lo tanto, puede reconocer directamente una señal de una imagen de entrada. Si las dos ramas dan respuestas unánimes, entonces se informa a un usuario de la señal reconocida. De lo contrario, se reinicia el proceso.



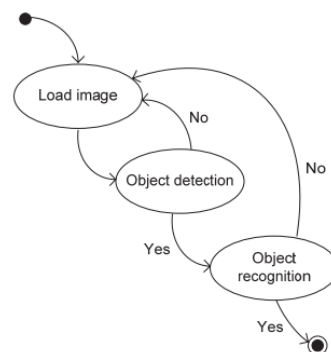
3. **Diagrama de interacción** - este tipo de diagrama debe utilizarse para modelar las dependencias temporales entre los mensajes enviados por los objetos participantes. Los factores clave son: actores participantes (objetos, componentes, etc.), secuencia de mensajes y tiempo.

En el ejemplo, hay tres participantes: usuario (u), software (s) y hardware (h). Un mensaje enviado por el usuario desencadena una serie de mensajes que se pasan entre el software y el hardware, y viceversa. Una vez que todos estos mensajes han sido procesados, se envía un mensaje final de vuelta al usuario.



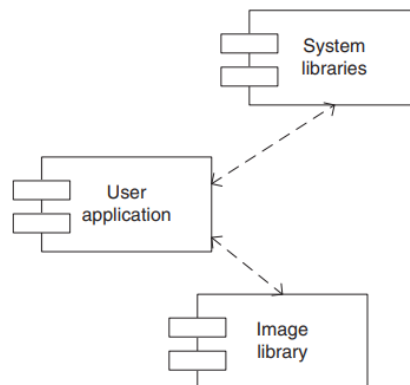
4. **Diagrama de transición de estados** - modela los estados y las transiciones que muestran la respuesta de un sistema a algunas excitaciones. El diagrama de estado modela los aspectos dinámicos de un sistema con estados y transiciones. Estos son elementos de las máquinas de estados, que pueden ser de diferentes tipos, como por ejemplo Mealy o Moore. En el primero, cada acción siguiente está gobernada por el estado actual y los valores de la entrada actual. En el segundo, las acciones dependen solo del estado actual. También pueden existir máquinas de estados que combinen ambas aproximaciones.

En el ejemplo, hay dos estados específicos: un estado inicial (representado por un punto único) y un estado final (representado por un punto doble). El estado siguiente se determina a partir del estado actual y la transición.



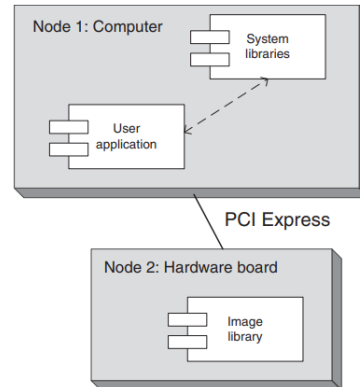
5. **Diagrama de componentes** - muestra las relaciones entre el software y los componentes externos.

En este diagrama, se visualiza una relación entre tres componentes. Los componentes son entidades de programación autosuficientes encapsuladas, como clases, paquetes, pequeños programas, etc. En el ejemplo proporcionado, estos son: aplicación del usuario, bibliotecas del sistema y biblioteca de imágenes. Todos deben estar conectados de alguna manera por medio de la aplicación del usuario.

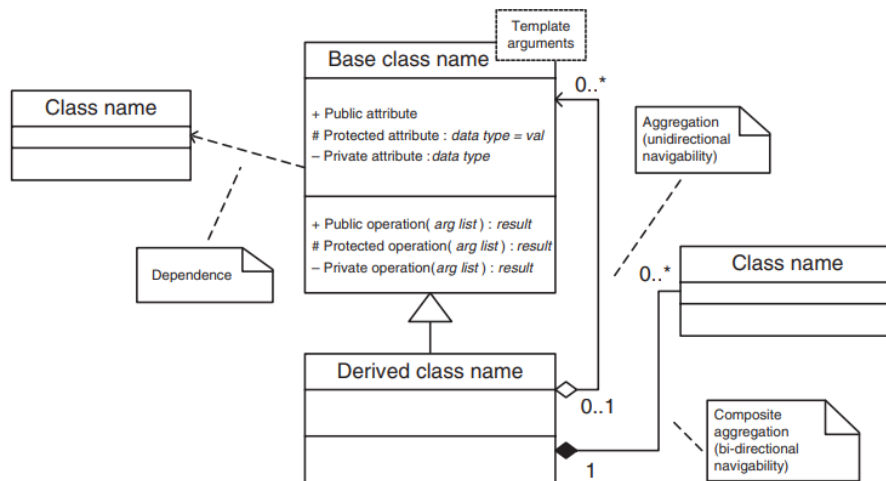


6. **Diagrama de despliegue** - muestra el despliegue y/o configuración de componentes en procesadores y dispositivos. Además de visualizar los componentes y sus conexiones, el diagrama de despliegue también considera los dispositivos, como microprocesadores, computadoras, etc., en los que se instalan los componentes. Estos dispositivos se llaman nodos.

En este diagrama, hay dos nodos, cada uno con sus propios componentes. Los nodos en este ejemplo están conectados mediante la conexión PCI Express.



7. **Diagrama de clases** - muestra la estructura estática, las relaciones y la estructura interna de los objetos. Este tipo de diagramas son los más utilizados. Con ellos se representan las jerarquías de clases y sus relaciones.



El concepto más básico es el de una clase con sus componentes. Una clase puede contener atributos (datos) y métodos (funciones). Cada uno de ellos puede pertenecer a la sección pública, protegida o privada de la clase. Los atributos públicos son accesibles para todos los usuarios de un objeto de esa clase, mientras que el uso de atributos protegidos está restringido exclusivamente a esta clase y todas las clases derivadas públicas o protegidas que se derivan de ella. Estas restricciones están modeladas por los signos +, # y -, respectivamente. Un triángulo pequeño representa una clase derivada de su clase base. El diamante vacío denota una agregación, lo que significa que una clase posee objetos que no son necesariamente miembros de esa clase. El diamante lleno significa una agregación compuesta, lo que indica que un objeto de una clase está compuesto por algunos otros objetos. También pueden existir algunas dependencias sueltas entre clases, que se denotan con una línea discontinua con una flecha.

Asegurar la corrección del código

La corrección del código está comprometida por numerosos tipos de errores. Los más comunes se deben a simples errores de programación o fragmentos de código no protegidos contra datos de entrada inválidos. La situación empeora si los problemas resultan de un diseño incorrecto.

Se aconseja tomar medidas preventivas desde el inicio del proceso de diseño para ayudar a escribir código que funcione correctamente y no se bloquee. El primer paso es analizar y entender el problema antes de comenzar a escribir el código. Se recomienda utilizar un enfoque de arriba hacia abajo o, en otras palabras, una estrategia de divide y vencerás.

Cada tarea superior debe dividirse en tareas más pequeñas. Luego, trabaje en cada tarea por separado, teniendo en cuenta la misma regla de divide y vencerás, y así sucesivamente. Sin embargo, no olvide la interacción común entre los módulos. Un enfoque sistemático y estructurado puede ayudar a garantizar que el código sea robusto, eficiente y fácil de mantener.

Algunas recomendaciones adicionales incluyen:

- Utilizar técnicas de depuración y pruebas para identificar y corregir errores
- Implementar mecanismos de validación de datos para prevenir entradas inválidas
- Seguir principios de diseño sólidos, como la separación de responsabilidades y la cohesión
- Utilizar herramientas de análisis de código estático y dinámico para detectar problemas potenciales

Un enfoque efectivo para asegurar la corrección de un programa es utilizar la técnica de programación por contrato [2]. Esto implica tratar un procedimiento software como si fuera un contrato comercial, con condiciones previas y posteriores bien definidas. Además, existen invariantes, es decir, reglas que deben ser verdaderas en cualquier punto de la ejecución del programa. De esta manera, cada módulo de software, componente, clase, método o incluso bloque de código puede tener sus propias condiciones previas y posteriores, así como invariantes.

Un enfoque simple, pero muy útil, es que los componentes chequeen todas las aserciones de corrección de código o requisitos. Para que un componente de software funcione correctamente, todas sus aserciones deben ser verdaderas durante la ejecución, antes y después de la ejecución del componente. Si no es así, decimos que una aserción se ha "disparado". En un programa, las aserciones son simplemente líneas de código que verifican la consistencia de los datos o condiciones que el programador considera que siempre deben ser verdaderas.

Las aserciones pueden incluir:

- Condiciones previas: que deben ser verdaderas antes de que se ejecute un método o bloque de código
- Condiciones posteriores: que deben ser verdaderas después de que se haya ejecutado un método o bloque de código

- Invariantes: que deben ser verdaderas en cualquier punto de la ejecución del programa

Al utilizar aserciones, "**assert**" en python, los programadores pueden garantizar que su código sea más robusto y menos propenso a errores. Además, las aserciones pueden ayudar a detectar problemas en una etapa temprana del desarrollo, lo que puede reducir el tiempo y el esfuerzo necesarios para depurar y solucionar errores.

```
import cv2
import numpy as np

class Imagen:
    """
    Clase que representa una imagen.

    Attributes:
        fRow (int): Número de filas de la imagen.
        fCol (int): Número de columnas de la imagen.
        fData (numpy.ndarray): Datos de la imagen.
    """

    def create(self, col, row):
        """
        Crea una imagen de zeros.

        :param col: Número de columnas de la imagen.
        :type col: int
        :param row: Número de filas de la imagen.
        :type row: int
        :return: None
        :rtype: None

        Nota:
            Los datos se inicializa con zeros.

        Raises:
            AssertionError: Si el número de columnas o filas es menor o igual a 0.
        """
        assert col > 0, "La cantidad de columnas debe ser mayor que 0"
        assert row > 0, "La cantidad de filas debe ser mayor que 0"

        self.fRow = row
        self.fCol = col
        self.fData = np.zeros((row, col), dtype=np.uint8)

        # Verificar si la imagen se creó correctamente
        assert self.fData is not None, "No se pudo crear la imagen"
        assert self.fData.shape == (row, col), "Dimensiones de imagen incorrectas"

    def mostrar_imagen(self):
        """
        Muestra la imagen utilizando OpenCV.

        :return: None
        :rtype: None
        """
        assert self.fData is not None, "La imagen no está definida"
        assert len(self.fData.shape) == 2, "La imagen no es correcta"
        cv2.imshow("Imagen", self.fData)
        cv2.waitKey(0)
        cv2.destroyAllWindows()

# Ejemplo de uso
imagen = Imagen()
imagen.create(512, 512)
imagen.mostrar_imagen()
```

Es importante destacar que las aserciones no reemplazan las pruebas unitarias o de integración, sino que complementan estas técnicas para asegurar la corrección y calidad

del código. Al combinar aserciones con otras técnicas de verificación, los programadores pueden desarrollar software más confiable y mantenible.

Depuración de errores

Al desarrollar un sistema, una buena estrategia es no posponer la depuración hasta que todos los componentes estén finalizados y conectados entre sí. En su lugar, se debe realizar la depuración en paralelo con el desarrollo de los diferentes componentes. Luego, también se debe verificar el sistema completo cuando todos sus módulos estén ensamblados juntos.

A veces, es imposible verificar todos los casos posibles de ejecución o datos de entrada, lo que puede llevar a malfuncionamientos peligrosos en software crítico. Un principio importante es depurar el código lo antes posible. Este concepto se facilita mediante el paradigma orientado a objetos de una clase autocontenida y la encapsulación de clases. Cada clase debe diseñarse de manera que resulte en una implementación clara con un estado y una interfaz bien definidos. Su dependencia de otros objetos también debe estar bien especificada. Esto crea un tipo de "espacio de restricciones" que se puede verificar por separado.

Inmediatamente después de la implementación, una clase debe ser depurada por su programador. Esta estrategia es recomendable, ya que, si se pospone la depuración, algunos detalles pueden volverse vagos, lo que hace que las pruebas sean aún más difíciles. Luego, si es posible, un componente de software también debe ser verificado por otra persona.

Al probar sistemas de visión, el principal problema es la gran variabilidad de los datos de entrada posibles. Generalmente no es posible verificar los algoritmos para todas las posibles imágenes de entrada. En su lugar, se puede crear un patrón de prueba del que se pueda predecir fácilmente la salida del algoritmo. A veces, una estrategia muy útil es crear patrones de "borde", es decir, ejemplos de datos de entrada para condiciones específicas de inicio o parada, como todos los píxeles negros, blancos o una cuadrícula, etc.

Al tratar con procedimientos iterativos, también es muy importante verificar sus condiciones de parada. Si no es posible verificar todos los casos posibles, un contador adicional con un límite preestablecido de iteraciones puede ser de ayuda.

La depuración temprana es una estrategia efectiva para garantizar la calidad y la confiabilidad del software. Al depurar el código lo antes posible, se reduce el riesgo de malfuncionamientos peligrosos y se mejora la calidad del código. Es importante utilizar herramientas de depuración adecuadas y seguir una estrategia sistemática para detectar y corregir errores.

Existen muchas herramientas de análisis de código estático y dinámico que pueden ayudar a detectar problemas potenciales en el código. A continuación, te menciono algunas de las más populares:

Análisis de código estático:

1. **SonarQube:** Una plataforma de análisis de código que proporciona informes detallados sobre la calidad del código, incluyendo problemas de seguridad, bugs y violaciones de normas de codificación.
2. **CodeCoverage:** Una herramienta que mide la cobertura del código, es decir, qué porcentaje del código se ejecuta durante las pruebas unitarias.
3. **PyLint:** Un analizador de código estático para Python que detecta problemas como errores de sintaxis, violaciones de normas de codificación y posibles bugs.
4. **Checkstyle:** Una herramienta que verifica si el código Java cumple con las normas de codificación establecidas.
5. **Cppcheck:** Un analizador de código estático para C++ que detecta problemas como errores de sintaxis, violaciones de normas de codificación y posibles bugs.

Análisis de código dinámico:

1. **JUnit:** Una framework de pruebas unitarias para Java que permite escribir y ejecutar pruebas automatizadas.
2. **PyUnit:** Un framework de pruebas unitarias para Python que permite escribir y ejecutar pruebas automatizadas.
3. **Valgrind:** Una herramienta que detecta problemas de memoria en código C++ y otros lenguajes, como fugas de memoria y acceso a memoria no inicializada.
4. **GDB:** Un depurador que permite analizar el comportamiento del código en tiempo de ejecución.
5. **New Relic:** Una herramienta que monitorea el rendimiento del código y detecta problemas de errores, lentitud y consumo excesivo de recursos.

Herramientas de análisis de seguridad:

1. **OWASP ZAP:** Un escáner de vulnerabilidades web que detecta problemas de seguridad como inyección SQL y cross-site scripting (XSS).
2. **Burp Suite:** Una herramienta que detecta problemas de seguridad en aplicaciones web, incluyendo vulnerabilidades como inyección SQL y XSS.
3. **Nessus:** Un escáner de vulnerabilidades que detecta problemas de seguridad en sistemas operativos, redes y aplicaciones.

Otras herramientas:

1. **CodePro AnalytiX:** Una plataforma de análisis de código que proporciona informes detallados sobre la calidad del código, incluyendo problemas de seguridad, bugs y violaciones de normas de codificación.
2. **Kiuwan:** Una plataforma de análisis de código que detecta problemas de seguridad, bugs y violaciones de normas de codificación en código Java, C++, Python y otros lenguajes.
3. **Codacy:** Una herramienta que analiza el código y proporciona informes detallados sobre la calidad del código, incluyendo problemas de seguridad, bugs y violaciones de normas de codificación.

Es importante mencionar que no hay una sola herramienta que pueda detectar todos los problemas potenciales en el código. Por lo tanto, es recomendable utilizar una combinación de herramientas para asegurarse de que el código sea lo más robusto y seguro posible.

Patrones de diseño

Los patrones de diseño son construcciones que exhiben comportamientos similares en diferentes aplicaciones. Existen varios tipos de patrones de diseño de software, clasificados en categorías como creación, estructura y comportamiento.

Aunque no son una receta para resolver todos los problemas de diseño de software, los patrones de diseño pueden ser muy útiles si se utilizan correctamente. Ayudan a identificar construcciones comunes en los sistemas, que pueden ser implementados de manera unificada. Esto conduce a diseños más comprensibles, componentes reutilizables y código más legible.

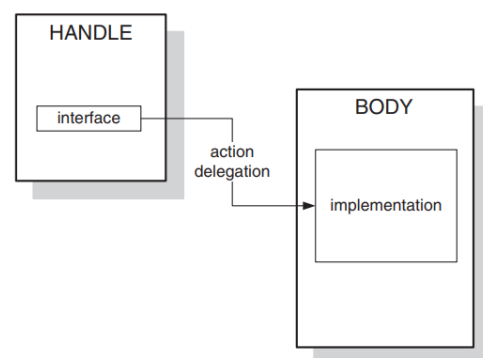
A continuación, se presentará información básica sobre patrones de diseño comúnmente utilizados en software de visión.

Interfaces

Los sistemas están compuestos por módulos que se comunican a través de interfaces. Por lo tanto, cambiar la especificación de una única interfaz provoca la variación de todas las partes del sistema que cooperan entre sí. Por lo tanto, las interfaces deben estar bien diseñadas antes de incorporarlas al sistema. Solo deberían cambiarse si es necesario. Sin embargo, detrás de las interfaces se encuentran algoritmos que operan sobre las estructuras de datos. A esta parte la llamamos el cuerpo de un sistema. Este sufre muchas variaciones, no solo durante la implementación, sino también cuando el sistema crece y cambia a lo largo de los años. Entender el comportamiento de estos dos ámbitos es importante para diseñar sistemas informáticos complejos. Por lo tanto, surge la pregunta sobre cómo podemos unir estas dos partes diferentes.

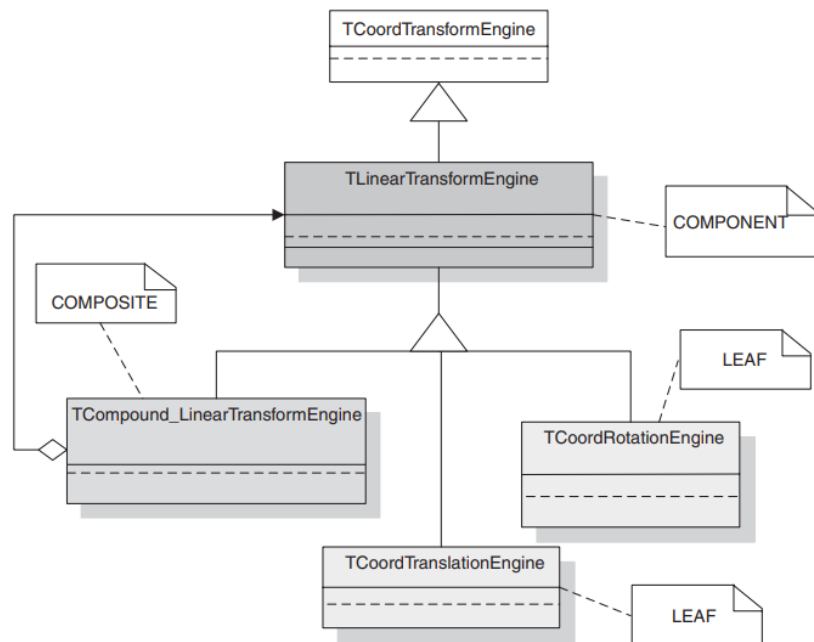
Una respuesta es dividir un diseño en dos líneas de desarrollo separadas. La primera se ocupa del diseño e implementación de las interfaces, y la segunda del cuerpo. En términos de patrones de diseño, la primera se llama mango (handle) y la segunda cuerpo (body).

Esta figura muestra la relación entre un mango (handle) y su cuerpo (body). El papel primordial del mango es definir la interfaz que se utiliza para comunicarse con otros componentes. Sin embargo, la ejecución real de una acción se delega a la parte del cuerpo asociada. El acoplamiento entre los dos es bastante laxo, por lo que el cuerpo puede cambiarse fácilmente.



Composición

La composición es uno de los patrones estructurales más comunes e interesantes. Permite la composición de objetos en estructuras similares a árboles que representan jerarquías parte-todo. Por lo tanto, desde un punto de vista externo, una sola hoja así como una composición de hojas pueden ser tratadas de la misma manera, es decir, exhiben la misma interfaz, aunque tengan estructuras internas diferentes.



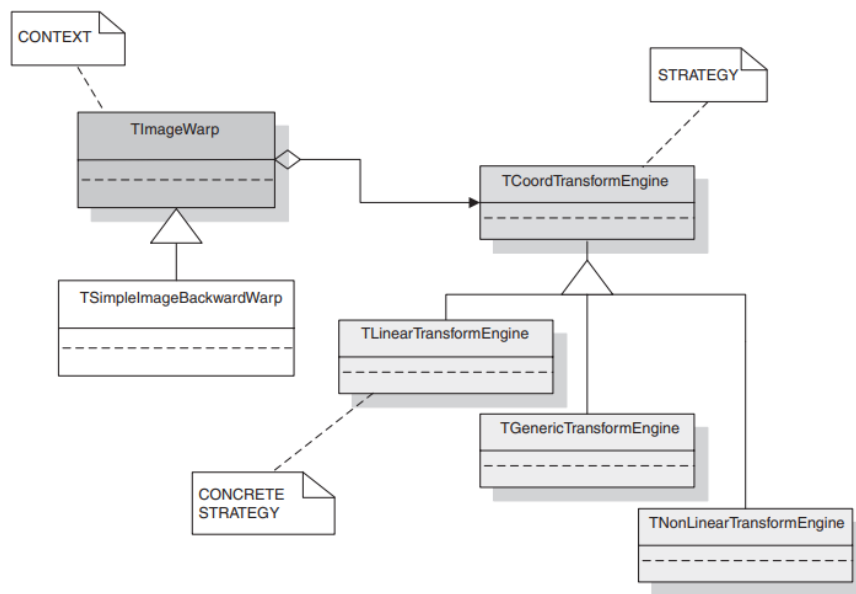
La figura anterior muestra una jerarquía de clases para la composición de transformaciones de coordenadas. Hay tres tipos de objetos involucrados: un componente, un compuesto y una hoja. El primero define una interfaz común y usualmente se implementa como una clase virtual pura, es decir, una que no sirve para instanciar objetos de su tipo. El compuesto y la hoja son hijos del mismo nivel. Sin embargo, el compuesto puede agregar una o más instancias de tales hijos, es decir, puede agregar hojas y/o otros compuestos, y así sucesivamente, construyendo estructuras recursivas. Por otro lado, desde el punto de vista de otros módulos, todos los hijos comparten las mismas propiedades, con una interfaz común definida en el componente (es decir, el `TLinearTransformEngine`).

En la figura, el `TLinearTransformEngine` es una clase base y define una interfaz común para un grupo de transformaciones lineales de coordenadas de imagen, como rotación, traslación y escalado. Estas están representadas por las hojas apropiadas, es decir, las clases `TCoordTranslationEngine` y `TCoordRotationEngine`. Sin embargo, una transformación lineal compuesta por rotación y escalado puede estar representada por una combinación de las hojas apropiadas. Por lo tanto, cualquier transformación lineal puede estar representada por el `TCompoundLinearTransformEngine`. La clase compuesta suele estar dotada de métodos para agregar, eliminar y acceder a sus componentes, es decir, las hojas.

El patrón de diseño de composición debe considerarse en los casos de estructuras compuestas jerárquicas que tienen una interfaz uniforme hacia el mundo exterior.

Estrategia

La estrategia denota un patrón que permite controlar la variabilidad de los algoritmos. Su parte integral constituye una interfaz que permite la aplicación uniforme de diferentes algoritmos. Los algoritmos pueden definirse de muchas maneras, por ejemplo como objetos función. Un algoritmo particular se elige en función de alguna información sobre los datos procesados.



La figura anterior muestra la estructura del patrón de diseño de estrategia en el contexto del módulo de transformación geométrica de imágenes. La base TImageWarp constituye un contexto. La parte de la estrategia se realiza en TCoordTransformEngine donde se asigna la estrategia concreta que, en el ejemplo presente, puede ser una de las tres subclases TLinearTransformEngine, TGenericTransformEngine y TNonLinearTransformEngine.

Por otra parte, una clase de contexto contiene una referencia a un objeto de estrategia. Todas las acciones contenidas en la semántica de este objeto de estrategia se delegan desde el contexto a la estrategia. Dependiendo de la transformación particular solicitada por un módulo externo, en realidad se elige una de las estrategias concretas hijas en tiempo de ejecución y se pasa como referencia al contexto (o se pasa directamente a su constructor). Luego, la estrategia elegida realiza todas las acciones desde el contexto.

Nota: A modo de ejemplo, en [poliformaT](#), os he dejado pautas utilizadas por una ingeniería real para programar en C++.

Referencias:

- [1] Booch, G., Rumbaugh, J. and Jacobson, I. (1999) The Unified Modeling Language. User Guide, Addison-Wesley Longman.
- [2] McConnell, S. (2004) Code Complete, 2nd edn, Microsoft Press.